LAPORAN PRAKTIKUM PEMROGRAMAN BACKEND LANJUT

Pertemuan 12 — Testing & Quality, API Documentation, dan Docker Deployment

Nama Mahasiswa	: **Rozan Aiman Ramadani**
NIM	: **434241069**
Program Studi	: D4 Teknik Informatika — Fakultas Vokasi, Universitas Airlangga
Mata Kuliah	: Praktikum Pemrograman Backend Lanjut (Semester 5)
Tautan Repository GitHub	: **https://github.com/RozanRamadani/tugas1_go**
Tanggal Pengujian	: 7 November 2026

D.1 Berita Acara Perbaikan (9 Bug Tersembunyi pada B1)

Berikut adalah dokumentasi lengkap 9 kesalahan yang ditanam di Bagian B1, metode pemeriksaan yang mengungkapnya, akar masalah, perbaikan yang diterapkan, serta bukti keberhasilannya:

No	Lokasi (File, Fungsi)	Gejala yang Diamati	Pemeriksaan yang Mengungkapnya	Akar Masalah	Perbaikan
1	`course_rules_test.go` `TestWithinCreditLimit`	Test tetap PASS walaupun logika diubah menjadi `<` (melewati batas tepat 24 SKS tidak tertangkap).	**Uji Mutasi T5**	Tabel uji hanya memuat kasus 0, 12, 22(+3), dan 24(+1). Tidak ada uji untuk kondisi batas tepat `taken + adding == 24` (misal 21 + 3).	Menambahkan skenario batas pada slice `tests` : `{name: "tepat di batas 24 SKS", taken: 21, adding: 3, want: true}` .
2	`review_rules_test.go` `TestNormalizeTags`	Kompilasi ditolak: fungsi test tidak memenuhi signature standar Go testing.	**Compiler**	Parameter testing ditulis tanpa pointer (`t testing.T`) alih-alih `t *testing.T` , melanggar kontrak framework test Go.	Mengubah signature fungsi menjadi `func TestNormalizeTags(t *testing.T)` .
3	`fakes_test.go` `fakeReviewRepo.FindByCourse`	Kompilasi gagal pada compile-time assertion: `cannot use (*fakeReviewRepo)(nil) as repository.ReviewRepository` .	**Compiler**	Method fake menggunakan signature usang `(ctx context.Context, courseID int)` , padahal interface `ReviewRepository` membutuhkan `(ctx context.Context, q model.ReviewQuery)` .	Menyesuaikan signature dan implementasi filtering query: `func (f *fakeReviewRepo) FindByCourse(_ context.Context, q model.ReviewQuery) ([]model.Review, error)` .
4	`review_service_test.go` `newTestEnv`	Test flaky dan gagal saat urutan diacak (`TestListReviews_NewCourseIsEmpty` menemukan data sisa test sebelumnya).	**T2 (`-shuffle=on`)**	Repository ulasan fake dideklarasikan sebagai variabel global (`var sharedReviews = newFakeReviewRepo()`), menyebabkan *state leakage* antar-test.	Menginstansiasi repository fake baru di dalam fungsi `newTestEnv(t)` menggunakan `reviews := newFakeReviewRepo()` .
5	`review_errors_test.go` `TestTranslateReviewError`	Kompilasi gagal: `cannot refer to unexported name service.translateReviewError` .	**Compiler**	File menggunakan `package service_test` (black-box), sehingga tidak dapat mengakses fungsi privat `translateReviewError` di package `service` .	Mengubah header package menjadi `package service` (white-box test).
6	`notification_worker_test.go` `TestHandleSend_...`	Kompilasi gagal: `not enough arguments in call to NewNotificationWorker` (5 argumen diberikan, butuh 6).	**Compiler**	Konstruktor `NewNotificationWorker` membutuhkan 6 parameter (termasuk mailer), tetapi hanya dioper 5 parameter.	Menambahkan argumen `nil` untuk parameter mailer yang tidak disentuh skenario kegagalan permanen.

No	Lokasi (File, Fungsi)	Gejala yang Diamati	Pemeriksaan yang Mengungkapnya	Akar Masalah	Perbaikan
7	`notification_worker_test.go` `TestHandleSend_...`	Test gagal: error SkipRetry dianggap tidak cocok karena dibungkus oleh layer lain (wrapped error).	**T1 (`go test ./...`)**	Pemeriksaan error menggunakan operator kesetaraan langsung `err != asynq.SkipRetry` yang tidak dapat mendeteksi wrapped error (`%w`).	Menggunakan fungsi unwrap standar Go: `if !errors.Is(err, asynq.SkipRetry)`.
8	`tests/integration/main_test.go` `TestMain`	Integration test selalu dilewati: mencetak `TEST_DATABASE_URL tidak diisi: integration test dilewati` padahal variabel terisi.	**T4**	Kondisi pengecekan terbalik: `if dsn != ""` membuat program langsung keluar (exit 0) saat DSN tersedia.	Memperbaiki logika percabangan menjadi `if dsn == ""`.
9	`tests/integration/main_test.go` `seedCourse`	Test gagal dengan `course not found` atau `enrolled_count = 0` karena baris kelas sudah terhapus sebelum assertion test berjalan.	**T4**	Pembersihan database di dalam helper `seedCourse` menggunakan `defer func()`, sehingga data dihapus seketika helper selesai dieksekusi (bukan saat test selesai).	Mengganti `defer func()` dengan `t.Cleanup(func() { ... })` agar pembersihan ditunda hingga seluruh lifecycle test selesai.

D.2 Menambah Test (Integration Tests & Mutation Testing)

Sesuai instruksi Tugas Mandiri D.2, telah ditambahkan dua skenario integration test pada `tests/integration/mandiri_test.go`:

1. Kode Integration Test Baru (`tests/integration/mandiri_test.go`)

```
//go:build integration
package integration_test

import (
	"context"
	"net/http"
	"net/http/httptest"
	"strconv"
	"strings"
	"testing"
	"latihan-fiber/app/model"
)

// D.2.1: Swap ke kelas yang kuotanya penuh harus ditolak 409 COURSE_FULL,
// mahasiswa tetap di kelas asal, enrolled_count tidak berubah, dan outbox kosong.
func TestEnrollEndpoint_SwapCourseFull(t *testing.T) {
	a := newAPI(t)
	student := seedStudent(t)
	targetStudent := seedStudent(t)

	courseA := seedCourse(t, 3, 40) // kelas asal
	courseB := seedCourse(t, 3, 1)  // kelas tujuan dengan kuota 1

	if status, _ := a.enroll(t, student, courseA.ID); status != http.StatusCreated {
		t.Fatalf("enroll kelas A: status = %d, seharusnya 201", status)
	}
	if status, _ := a.enroll(t, targetStudent, courseB.ID); status != http.StatusCreated {
		t.Fatalf("enroll target student kelas B: status = %d, seharusnya 201", status)
	}

	token, _ := a.jwt.GenerateAccessToken(model.AuthUser{UserID: student.ID, Username: student.Username, Role: "user"})
	body := `{"from_course_id":` + strconv.Itoa(courseA.ID) + `,"to_course_id":` + strconv.Itoa(courseB.ID) + `}`
	req := httptest.NewRequest(http.MethodPost, "/api/v1/enrollments/swap", strings.NewReader(body))
	req.Header.Set("Authorization", "Bearer "+token)
	req.Header.Set("Content-Type", "application/json")

	resp, err := a.app.Test(req, -1)
	if err != nil {
		t.Fatalf("request swap: %v", err)
	}
	defer resp.Body.Close()

	if resp.StatusCode != http.StatusConflict {
		t.Errorf("status swap = %d, seharusnya 409 Conflict", resp.StatusCode)
	}

	enrolledA := countRows(t, `SELECT COUNT(*) FROM enrollments WHERE user_id = $1 AND course_id = $2`, student.ID, courseA.ID)
	if enrolledA != 1 {
		t.Errorf("mahasiswa harus tetap terdaftar di kelas asal (A)")
	}

	cA := countRows(t, `SELECT enrolled_count FROM courses WHERE id = $1`, courseA.ID)
	cB := countRows(t, `SELECT enrolled_count FROM courses WHERE id = $1`, courseB.ID)
	if cA != 1 || cB != 1 {
		t.Errorf("enrolled_count berubah: cA=%d, cB=%d, seharusnya tetap 1", cA, cB)
	}

	swappedEvents := countRows(t, `SELECT COUNT(*) FROM outbox WHERE (payload->>'user_id')::INT = $1 AND event_type = $2`,
		student.ID, model.EventEnrollmentSwapped)
	if swappedEvents != 0 {
		t.Errorf("event enrollment.swapped = %d, seharusnya 0", swappedEvents)
	}
}

// D.2.2: Drop kelas berhasil menghasilkan 204 No Content, baris enrollment hilang,
// enrolled_count berkurang satu, dan outbox mencatat event enrollment.dropped.
func TestEnrollEndpoint_DropSuccess(t *testing.T) {
	a := newAPI(t)
	student := seedStudent(t)
	course := seedCourse(t, 3, 40)

	if status, _ := a.enroll(t, student, course.ID); status != http.StatusCreated {
```

```
                t.Fatalf("enroll gagal: status = %d", status)
        }

        token, _ := a.jwt.GenerateAccessToken(model.AuthUser{UserID: student.ID, Username: student.Username, Role: "user"})
        req := httptest.NewRequest(http.MethodDelete, "/api/v1/courses/"+strconv.Itoa(course.ID)+"/enroll", nil)
        req.Header.Set("Authorization", "Bearer "+token)
        req.Header.Set("Content-Type", "application/json")

        resp, err := a.app.Test(req, -1)
        if err != nil {
                t.Fatalf("request drop: %v", err)
        }
        defer resp.Body.Close()

        if resp.StatusCode != http.StatusNoContent {
                t.Fatalf("status drop = %d, seharusnya 204", resp.StatusCode)
        }

        enrolled := countRows(t, `SELECT COUNT(*) FROM enrollments WHERE user_id = $1 AND course_id = $2`, student.ID, course.ID)
        if enrolled != 0 {
                t.Errorf("baris enrollment = %d, seharusnya 0 setelah drop", enrolled)
        }

        count := countRows(t, `SELECT enrolled_count FROM courses WHERE id = $1`, course.ID)
        if count != 0 {
                t.Errorf("enrolled_count = %d, seharusnya 0 setelah drop", count)
        }

        dropEvents := countRows(t, `SELECT COUNT(*) FROM outbox WHERE (payload->>'user_id')::INT = $1 AND event_type = $2`,
                student.ID, model.EventEnrollmentDropped)
        if dropEvents != 1 {
                t.Errorf("event enrollment.dropped = %d, seharusnya tepat 1", dropEvents)
        }
}
```

2. Bukti Uji Mutasi Test D.2

Uji Mutasi	File & Mutasi Kode Production	Hasil Pengujian (Bukti Test Gagal / Alarm Berbunyi)	Status
Mutasi Skenario 1 (`TestEnrollEndpoint_SwapCourseFull`)	`app/service/course_service.go` : Pada method `Swap` , bypass validasi kuota tujuan dengan mengganti pengecekan kuota (`repo.FindByID(toID)`).	`=== RUN TestEnrollEndpoint_SwapCourseFull mandiri_test.go:40: status swap = 200, seharusnya 409 Conflict --- FAIL: TestEnrollEndpoint_SwapCourseFull (0.04s) FAIL`	**TERBUKTI PEKA (KILLED)**
Mutasi Skenario 2 (`TestEnrollEndpoint_DropSuccess`)	`app/service/course_service.go` : Pada method `Drop` , mutasi fungsi dekrementasi dengan menghapus pemanggilan `courses.DecrementEnrolled` .	`=== RUN TestEnrollEndpoint_DropSuccess mandiri_test.go:81: enrolled_count = 1, seharusnya 0 setelah drop --- FAIL: TestEnrollEndpoint_DropSuccess (0.02s) FAIL`	**TERBUKTI PEKA (KILLED)**

D.3 Melengkapi Dokumentasi API (OpenAPI / Swagger)

Seluruh endpoint yang disyaratkan pada Bagian B2 dan Tugas Mandiri D.3 telah dilengkapi dengan anotasi deklaratif Swagger lengkap (summary, tags, param, success/failure response models, dan security header). Tidak ada lagi response yang menggunakan `fiber.Map` tanpa tipe eksplisit.

Daftar Lengkap 20 Endpoint yang Terdaftar di `docs/swagger.json` :

No	Tag	HTTP Method & Path	Deskripsi Ringkas / Ringkasan Anotasi	Security
1	`auth`	`POST /auth/register`	Mendaftarkan akun baru (role default "user")	Public
2	`auth`	`POST /auth/login`	Login dan mendapatkan TokenPair (access + refresh)	Public
3	`auth`	`POST /auth/refresh`	Menukar refresh token (rotation)	Public
4	`auth`	`POST /auth/logout`	Mencabut refresh token pengguna	BearerAuth
5	`auth`	`GET /auth/me`	Profil akun pengguna yang sedang login	BearerAuth
6	`courses`	`GET /courses`	Daftar seluruh mata kuliah yang tersedia	BearerAuth
7	`courses`	`GET /courses/mine`	Daftar KRS milik mahasiswa yang sedang login	BearerAuth
8	`courses`	`GET /courses/{id}`	Detail lengkap satu kelas berdasarkan ID	BearerAuth
9	`courses`	`POST /courses/{id}/enroll`	Mendaftar satu kelas (transaksional + outbox)	BearerAuth
10	`courses`	`DELETE /courses/{id}/enroll`	Keluar dari kelas (drop mata kuliah)	BearerAuth
11	`courses`	`POST /enrollments/swap`	Pindah kelas secara atomik	BearerAuth
12	`courses`	`PATCH /courses/{id}`	Mengubah kuota kelas (Optimistic Locking admin)	BearerAuth
13	`reviews`	`POST /courses/{id}/reviews`	Menulis ulasan mata kuliah (khusus peserta)	BearerAuth
14	`reviews`	`GET /courses/{id}/reviews`	Daftar ulasan kelas dengan cursor pagination	BearerAuth
15	`reviews`	`GET /courses/{id}/reviews/summary`	Ringkasan statistik rating ulasan kelas	BearerAuth
16	`reviews`	`GET /reviews/{id}`	Detail satu ulasan mata kuliah	BearerAuth
17	`reviews`	`POST /reviews/{id}/helpful`	Memberikan vote bermanfaat pada ulasan	BearerAuth
18	`reviews`	`POST /reviews/{id}/replies`	Membalas ulasan mahasiswa (reply limit)	BearerAuth
19	`users`	`GET /users`	Daftar seluruh user terdaftar (admin RBAC)	BearerAuth
20	`admin`	`GET /admin/queue`	Status antrean outbox dan redis queue monitor	BearerAuth

> **Bukti Verifikasi Dokumen Swagger:** Perintah `go tool swag init` menghasilkan dokumen `docs/docs.go`, `docs/swagger.json`, dan `docs/swagger.yaml` secara sempurna tanpa warning maupun error tipe data. Swagger UI dapat diakses secara langsung pada `http://localhost:8080/swagger/index.html` (Docker) maupun `http://localhost:3000/swagger/index.html` (Lokal).

D.4 Analisis Singkat

1. Coverage vs. Mutation Testing pada Kasus `TestWithinCreditLimit`

Mengapa coverage tidak dapat menggantikan uji mutation?
Coverage (khususnya *statement/line coverage*) hanya mencatat apakah sebuah baris kode *pernah dieksekusi* saat test suite berjalan, namun sama sekali tidak menilai apakah logika, batas kondisi tepi, dan hasil eksekusi tersebut diverifikasi dengan teliti melalui assertion. Sebagai contoh nyata, fungsi `WithinCreditLimit(taken, adding int) bool { return taken + adding <= 24 }` hanya terdiri dari satu baris logika. Jika test hanya menyertakan skenario `taken: 0, adding: 3`, coverage fungsi tersebut langsung tercatat **100.0%**. Namun, apabila operator diubah menjadi cacat logika (`<` alih-alih `<=`), test tersebut tetap lulus (PASS). Uji mutasi membuktikan bahwa coverage 100% dapat memberikan rasa aman palsu jika tidak diuji dengan mutasi kode produksi.

Mengapa uji mutation tidak dapat menggantikan coverage?
Uji mutasi hanya dapat diterapkan secara efektif pada kode yang sudah memiliki test awal. Jika suatu bagian kode atau percabangan sama sekali belum pernah dieksekusi oleh test (coverage 0%), mutasi pada bagian tersebut tidak ada gunanya karena pasti tidak ada test yang mengujinya. Selain itu, uji mutasi memerlukan biaya komputasi yang sangat tinggi (harus mengompilasi dan mengeksekusi test berulang kali untuk setiap mutasi sintaks). Coverage memberikan peta awal yang cepat untuk mengidentifikasi bagian kode mana yang belum tersentuh (blind spot), sedangkan mutation testing menguji ketajaman kualitas pengujian pada area yang telah tersentuh. Keduanya bersifat komplementer.

2. Keterbatasan Fake Repository Dibandingkan MongoDB Sungguhan

Perilaku MongoDB sungguhan yang TIDAK ditiru fake:
Fake repository pada Langkah 2 meniru unique index dengan perulangan linier di slice memory Go yang dilindungi oleh `sync.Mutex` lokal. Perilaku MongoDB sungguhan yang **tidak ditiru** adalah penegakan *Distributed Unique Index* di level storage engine WiredTiger saat menerima konkurensi request tinggi. Pada MongoDB sungguhan, duplikasi index memicu error driver spesifik (kode error `11000 duplicate key error`) dengan detail key pattern tertentu, serta memiliki karakteristik latensi I/O jaringan, transaksi multi-dokumen, partial filter expressions, dan perilaku connection pooling.

Jenis test yang seharusnya memeriksanya:
Integration Test terhadap instans MongoDB sungguhan (atau database test container). Integration test menghubungkan repository ke instance MongoDB riil, memastikan indeks unik dibuat dengan benar (`CreateOne(ctx, IndexModel{...})`), dan menguji bahwa operasi insert konkuren yang bertabrakan benar-benar menghasilkan error driver 11000 yang berhasil diterjemahkan oleh repository menjadi `repository.ErrDuplicate` .

3. Perbandingan Mekanisme Migrasi Docker vs. In-App Migrator (Multi-Replika)

Perbandingan:
Pada Docker Compose bawaan modul, migrasi dijalankan melalui volume binding `/docker-entrypoint-initdb.d` yang hanya dieksekusi *satu kali* saat volume PostgreSQL masih kosong. Sebaliknya, in-app migration (seperti pada `TestMain`) mengeksekusi DDL migration secara terprogram setiap kali aplikasi menyala.

Kelebihan dan Risiko jika API dijalankan 3 replika sekaligus:
• **Kelebihan:** Otomatisasi deployment sangat tinggi. Setiap container baru langsung memastikan skema database mutakhir tanpa memerlukan intervensi manual dari administrator database.
• **Risiko Kritis:** *Race Condition DDL dan Deadlock*. Ketika ketiga replika API menyala bersamaan, ketiganya akan berusaha mengeksekusi script DDL migrasi (`CREATE TABLE` , `ALTER TABLE` , `CREATE INDEX`) ke database PostgreSQL yang sama secara paralel. Tanpa mekanisme **Distributed Lock** (seperti `pg_advisory_lock`), migrasi dapat gagal di tengah jalan dengan error *tuple concurrently updated* atau skema corrupt. Solusi industri terbaik adalah menjalankan migrasi sebagai *init-container / migration job* terpisah sebelum replika API dinyalakan.

4. Risiko Keamanan Swagger UI Terbuka di Server Kampus & Port 8080

Dua Risiko Konkret:
1. **Information Disclosure & Attack Surface Reconnaissance:** Swagger UI mengekspos rincian seluruh endpoint privat (seperti `/admin/queue` , `/users/:id/role`), struktur payload JSON, tipe parameter, dan format token autentikasi. Penyerang di internet tidak perlu melakukan *blind probing*; mereka mendapatkan blueprint arsitektur backend secara cuma-cuma untuk merancang eksploitasi payload yang presisi.
2. **Brute Force & Unrestricted API Abuse:** Port 8080 diekspos langsung ke internet tanpa melewati Web Application Firewall (WAF) atau reverse proxy. Endpoint `/auth/login` dan `/auth/register` rentan diserang *credential stuffing* atau serangan Denial of Service (DoS) yang dapat menguras resource container backend dan koneksi database pool.

Perbaikan yang Tepat:
1. Nonaktifkan Swagger UI pada environment production/public dengan menyetel `SWAGGER_ENABLED=false` pada file environment server. Jika dokumentasi tetap dibutuhkan oleh pengembang, pasang autentikasi lapis dua (HTTP Basic Auth) atau batasi akses Swagger hanya untuk subnet IP jaringan internal kampus (VPN Kampus).
2. Tempatkan Reverse Proxy (seperti Nginx, Caddy, atau Cloudflare WAF) di depan port aplikasi untuk menerapkan enkripsi TLS/HTTPS (port 443), rate limiting ketat pada endpoint otentikasi, dan memblokir akses publik ke path `/admin/*` serta `/swagger/*` .

LAMPIRAN: BUKTI KELUARAN PENGUJIAN LENGKAP

1. Pemeriksaan C.1 (Testing Suite & Uji Mutasi)

T1 — Seluruh Unit Test (Semua Package):

```
$ go test -count=1 ./...
ok      latihan-fiber/app/service       0.390s
PASS
```

T2 — Pengujian Acak Bebas Data Leakage (Diulang 3 Kali):

```
$ go test -shuffle=on -count=5 ./app/service/
ok      latihan-fiber/app/service       0.406s
$ go test -shuffle=on -count=5 ./app/service/
ok      latihan-fiber/app/service       0.374s
$ go test -shuffle=on -count=5 ./app/service/
ok      latihan-fiber/app/service       0.351s
```

T3 — Cakupan Kode (Coverage 100% pada Aturan Murni):

```
$ go test -coverprofile coverage.out ./app/service/; go tool cover -func coverage.out
latihan-fiber/app/service/course_rules.go:13:         WithinCreditLimit       100.0%
latihan-fiber/app/service/course_rules.go:18:         CreditsAfterSwap        100.0%
latihan-fiber/app/service/notification_rules.go:13:   BuildMessage            100.0%
latihan-fiber/app/service/review_rules.go:11:         NormalizeTags           100.0%
```

T4 — Seluruh Integration Test (6 Kasus Dasar Modul + 2 Tugas Mandiri):

```
$ go test -tags=integration -count=1 -v ./tests/integration/
=== RUN   TestIncrementEnrolled_StopsAtQuota
--- PASS: TestIncrementEnrolled_StopsAtQuota (0.01s)
=== RUN   TestInsertEnrollment_Duplicate
--- PASS: TestInsertEnrollment_Duplicate (0.02s)
=== RUN   TestWithTx_RollsBackEveryStep
--- PASS: TestWithTx_RollsBackEveryStep (0.01s)
=== RUN   TestClaim_OnlyOnce
--- PASS: TestClaim_OnlyOnce (0.00s)
=== RUN   TestEnrollEndpoint_WritesOutboxOnlyOnSuccess
--- PASS: TestEnrollEndpoint_WritesOutboxOnlyOnSuccess (0.01s)
=== RUN   TestEnrollEndpoint_ConcurrentCreditLimit
--- PASS: TestEnrollEndpoint_ConcurrentCreditLimit (0.27s)
=== RUN   TestEnrollEndpoint_SwapCourseFull
--- PASS: TestEnrollEndpoint_SwapCourseFull (0.03s)
=== RUN   TestEnrollEndpoint_DropSuccess
--- PASS: TestEnrollEndpoint_DropSuccess (0.01s)
PASS
ok      latihan-fiber/tests/integration 0.932s
```

Uji Mutasi T5 (course_rules.go `<=` diganti `<`):

```
--- FAIL: TestWithinCreditLimit (0.00s)
    --- FAIL: TestWithinCreditLimit/tepat_di_batas_24_SKS (0.00s)
        course_rules_test.go:30: WithinCreditLimit(21, 3) = false, seharusnya true
FAIL
```

Uji Mutasi T6 (Hapus `FOR UPDATE` pada LockStudent — Diulang 3 Kali):

```
$ go test -tags=integration -count=3 -run Concurrent ./tests/integration/
--- FAIL: TestEnrollEndpoint_ConcurrentCreditLimit (0.22s)
    enroll_api_test.go:132: berhasil = 10, seharusnya 8 (hasil lengkap: map[201:10])
    enroll_api_test.go:136: baris enrollments = 10, seharusnya 8
FAIL (Terbukti gagal pada ketiga perulangan tanpa exception)
```

Uji Mutasi T7 (Hapus `ON CONFLICT` pada Claim Notification):

```
$ go test -tags=integration -count=1 -run Claim ./tests/integration/
--- FAIL: TestClaim_OnlyOnce (0.01s)
    notification_repository_test.go:38: claim kedua gagal: ERROR: duplicate key value violates unique constraint "notifications_outbox_id
```

```
_key" (SQLSTATE 23505)
FAIL
```

2. Pemeriksaan C.2 (API Documentation & Swagger UI)

```
$ go tool swag init
2026/11/07 10:20:00 Generate swagger docs....
2026/11/07 10:20:00 Generate general API Info, endpoint: main.go
2026/11/07 10:20:01 Successfully generated documentation at docs/

$ curl -I http://localhost:8080/swagger/index.html
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 3658
```

3. Pemeriksaan C.3 (Docker Deployment & Compose Verifications)

```
# Verifikasi Penolakan Secret Kosong:
$ docker compose --env-file nul config
error while interpolating services.api.environment.JWT_SECRET: required variable JWT_SECRET is missing a value: JWT_SECRET wajib diisi di
.env

# Status Layanan Docker Compose:
$ docker compose ps
NAME                      IMAGE                COMMAND                  SERVICE    STATUS
latihan-fiber-api-1       latihan-fiber:local  "./app -role=api"        api        Up (healthy)   0.0.0.0:8080->3000/tcp
latihan-fiber-mongo-1     mongo:9.0            "docker-entrypoint.s…"   mongo      Up (healthy)   27017/tcp
latihan-fiber-postgres-1  postgres:16-alpine   "docker-entrypoint.s…"   postgres   Up (healthy)   5432/tcp
latihan-fiber-redis-1     redis:7.4-alpine     "docker-entrypoint.s…"   redis      Up (healthy)   6379/tcp
latihan-fiber-worker-1    latihan-fiber:local  "./app -role=worker"     worker     Up             3000/tcp

# Health Endpoint API:
$ curl http://localhost:8080/api/v1/health
{"success":true,"message":"server, PostgreSQL, dan MongoDB berjalan"}

# Verifikasi Hak Akses User Non-Root:
$ docker compose exec api whoami
app

# Verifikasi Konten Direktori Runtime (Bebas .env dan Source Code):
$ docker compose exec api ls -a /app
.
..
app
logs

# Verifikasi Alur Notifikasi Asinkron:
$ curl -s http://localhost:8080/api/v1/notifications/mine -H "Authorization: Bearer <token>"
{"success":true,"message":"notifikasi berhasil diambil","data":[{"id":1,"channel":"email","subject":"KRS: berhasil terdaftar di IF101 Pem
rograman Backend Lanjut","body":"Halo mhs_rozan, Anda tercatat sebagai peserta kelas IF101 Pemrograman Backend Lanjut."}]}

# Verifikasi Persistensi Volume Data Setelah compose down dan up -d:
$ docker compose down && docker compose up -d
$ curl -s http://localhost:8080/api/v1/courses/mine -H "Authorization: Bearer <token>"
{"success":true,"message":"KRS berhasil diambil","data":{"courses":[{"id":1,"code":"IF101","name":"Pemrograman Backend Lanjut","credits":
3,"quota":40,"enrolled_count":1}],"total_credits":3,"max_credits":24}}
```